Parallel Programming Applications

From Serial Baselines to Parallel Speedups

Karen Cardiel Olea
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2209039@upy.edu.mx

Jorge Ramiro Chay Koyoc
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309052@upy.edu.mx

Angeles Alejandra Cruz Legorreta
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309064@upy.edu.mx

Diego Jesús Loría Campos
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309140@upy.edu.mx

Brad Robles García
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309198@upy.edu.mx

Elisabet Arelly Sulú Vela
Universidad Politécnica de Yucatán
Km. 4.5. Carretera Mérida — Tetiz
Tablaje Catastral 4448. CP 97357
Ucú, Yucatán. México
Email: 2309212@upy.edu.mx

***Abstract*—This report presents the development and analysis of four parallel programming applications implemented in High Performance Computing (HPC) environments. The exercises cover parallel matrix multiplication with multiple decomposition strategies, parallel cell image processing and morphological characterization using the DIC-C2DH-HeLa microscopy dataset, a forest fire cellular automaton driven by real-time NASA FIRMS satellite hotspot data using MPI domain decomposition, and a distributed-memory parallel K-Means clustering algorithm applied to the Covertype geographic dataset. For each exercise, serial baseline implementations are compared against parallel alternatives, analyzing speedup, efficiency, communication overhead, and practical limitations. Collective results demonstrate that effective parallelization depends critically on balancing computational workload against synchronization and communication costs, memory access patterns, and the granularity of the task decomposition strategy.**

***Index Terms*—HPC, Parallel Computing, Matrix Multiplication, MPI, Multiprocessing, Image Segmentation, Cellular Automaton, K-Means, Scalability, Distributed Computing, mpi4py, Data Science.**

I Introduction

Parallel programming is a fundamental approach in high performance computing because it allows computational workloads to be distributed across multiple processing units, reducing execution time and enabling the solution of larger and more demanding problems. In scientific computing, data analysis, and artificial intelligence, many relevant tasks exhibit structures that can benefit from either **shared-memory** or **distributed-memory parallelism**. However, the actual gain obtained from parallelization depends not only on the number of workers used, but also on the *granularity of the workload, communication overhead, memory access patterns, synchronization cost, and load balancing*.

This report presents the development and analysis of four applications of parallel programming. The assignment requires a serial baseline and at least one parallel implementation for each exercise, together with reproducible performance measurements and an interpretation of *speedup, efficiency, and technical limitations*. In addition, the report is organized by exercise and includes, for each one, evidence of completeness, results, and discussion.

The four exercises cover distinct application domains. Exercise 1 studies *parallel matrix multiplication* using several decomposition strategies for dense and sparse inputs. Exercise 2 focuses on *microscopy image processing* and morphological characterization of cells, where the same workflow can be applied independently to multiple images. Exercise 3 models *forest fire propagation with a cellular automaton* initialized from NASA FIRMS hotspot detections and accelerated with MPI. Exercise 4 *analyzes the parallelization of K-means clustering* on the Covertype dataset using distributed-memory communication. Together, these problems illustrate different forms of parallelism and highlight the trade-offs between computational work, data movement, and synchronization.

II Exercise 1: Parallel Matrix Multiplication and Memory Access

II-A Methodology

Matrix multiplication is a classical high-performance computing problem because it is widely used in scientific computing, simulation, and machine learning. In this exercise, several dense and sparse matrix multiplication strategies were implemented and compared in order to analyze how different decomposition methods affect runtime, scalability, and efficiency.

To ensure the reproducibility and portability of the experiments, the complete solution was packaged within a Docker container (`proyecto-hpc`). The source code was designed with a modular architecture and includes a main Python orchestrator (`run_all.py`)

that sequentially executes six scripts addressing the requested tasks: `baseline.py`, `parallel_mp.py`, `parallel_mp2.py`, `parallel_mpi.py`, `sparse_experiments.py`, and `strassen.py`.

A serial baseline implementation was first developed and validated using small matrices to ensure correctness, empirically confirming the theoretical $\mathcal{O}(n^3)$ time complexity. After validation, multiple parallel approaches were explored:

- **Row-based multiprocessing:** Partitions matrix A into contiguous row blocks, assigning each block to a worker process.
- **Column-based multiprocessing:** Evaluates how a different one-dimensional partitioning strategy impacts memory access patterns and cache behavior.
- **Block-based (2D) decomposition:** Distributes computation across workers using two-dimensional tiles.
- **Distributed-memory MPI:** Uses `mpi4py` with `bcast`, `scatter`, and `gather` operations to analyze interprocess communication overhead.
- **Strassen's algorithm:** Both a pure recursive version and a hybrid variant (switching to NumPy BLAS at a configurable threshold) were compared against an optimized NumPy baseline [1].

To complement dense matrix experiments, sparse matrices from the SuiteSparse Matrix Collection [3], [4] were analyzed to highlight how sparsity and static load division affect performance.

```
================================================
   EJECUTANDO TODAS LAS PRUEBAS DEL EJERCICIO 1
================================================



------------------------------------------------------------
-> 1. Ejecutando Baseline Serial O(n^3)...
------------------------------------------------------------

Iniciando experimentos seriales...
Tamaño: 64x64 | Tiempo: 0.0578s | Correcto: True
Tamaño: 128x128 | Tiempo: 0.3134s | Correcto: True
Tamaño: 256x256 | Tiempo: 2.5969s | Correcto: True

Resumen de resultados:
   Method  Matrix_Size  Execution_Time_s  Correct
0  Serial           64          0.057794     True
1  Serial          128          0.313384     True
2  Serial          256          2.596890     True

------------------------------------------------------------
-> 2. Ejecutando Multiprocessing (Filas, Columnas, Bloques)...
------------------------------------------------------------

Generando matrices de 256x256...

Ejecutando partición por FILAS con 10 workers...
Ejecutando partición por COLUMNAS con 10 workers...

Resumen de resultados paralelos:
         Method  Matrix_Size  Workers  Execution_Time_s  Correct
0  Parallel_Rows          256       10          0.108051     True
1  Parallel_Cols          256       10          0.118436     True

------------------------------------------------------------
-> 2. Ejecutando Multiprocessing Actualizado (Filas, Columnas, Bloques)...
------------------------------------------------------------

Generando matrices de 512x512...

Resumen de resultados (Multiprocessing):
        Method  Size  Workers    Time_s
0         Rows   512       10  0.200511
1         Cols   512       10  0.144282
2  Blocks (2D)   512        9  0.213932

------------------------------------------------------------
-> 3. Ejecutando Memoria Distribuida (MPI - 4 Workers)...
------------------------------------------------------------

[Rank 0] Generando matrices de 512x512 para 4 procesos...

=== Resumen de Resultados (MPI) ===
Método      : MPI (Distributed Memory)
Workers     : 4
Tiempo (s)  : 0.004226
Correcto    : True

Nota para el reporte: Este tiempo incluye el overhead de serializar
y enviar/recibir mensajes a través de MPI (bcast, scatter, gather).

------------------------------------------------------------
-> 4. Ejecutando Análisis de Matrices Dispersas (Sparsity)...
------------------------------------------------------------


============================================
Analizando Matriz: bcspwr08.mtx
============================================
(Tiempo de carga: 0.0104s)

[Metadatos de la Matriz]
Tamaño                : 1624x1624
Elementos Totales     : 2637376
Elementos No Cero (nnz) : 6050
Porcentaje de ceros   : 99.77% (Sparsity)

[Rendimiento]
Tiempo Multiplicación Dispersa (CSR) : 0.000396s

[Análisis de Desequilibrio de Carga - Partición por 4 Filas]
Worker 0 (Filas 0- 405) : Recibe  1471 operaciones (24.3% de la carga total)
Worker 1 (Filas 406- 811) : Recibe  1540 operaciones (25.5% de la carga total)
Worker 2 (Filas 812-1217) : Recibe  1573 operaciones (26.0% de la carga total)
Worker 3 (Filas 1218-1623) : Recibe  1466 operaciones (24.2% de la carga total)

============================================
Analizando Matriz: bcsstk18.mtx
============================================
(Tiempo de carga: 0.0155s)

[Metadatos de la Matriz]
Tamaño                : 11948x11948
Elementos Totales     : 142754704
Elementos No Cero (nnz) : 149090
Porcentaje de ceros   : 99.90% (Sparsity)

[Rendimiento]
Tiempo Multiplicación Dispersa (CSR) : 0.013589s

[Análisis de Desequilibrio de Carga - Partición por 4 Filas]
Worker 0 (Filas 0-2986) : Recibe 31208 operaciones (20.9% de la carga total)
Worker 1 (Filas 2987-5973) : Recibe 38553 operaciones (25.9% de la carga total)
Worker 2 (Filas 5974-8960) : Recibe 39945 operaciones (26.8% de la carga total)
Worker 3 (Filas 8961-11947) : Recibe 39384 operaciones (26.4% de la carga total)

============================================
Conclusión para el reporte:
Si el porcentaje de carga (% de operaciones) varía mucho entre workers,
sufrimos de 'Load Imbalance'. El worker con más carga será el cuello de botella,
mientras los demás se quedan ociosos esperando a que termine.
============================================


------------------------------------------------------------
-> 5. Ejecutando Algoritmo Híbrido de Strassen...
------------------------------------------------------------

Generando matrices densas de 512x512 para prueba Strassen...

Ejecutando Strassen Puro (Esto tomará unos segundos por el overhead)...
Ejecutando Strassen Híbrido...

=== Resumen de Resultados (Strassen) ===
                    Method    Time_s
0            NumPy Baseline  0.001242
1     Strassen Puro (th=16)  0.089204
2  Strassen Híbrido (th=128)  0.005340

Exactitud de los cálculos: True

================================================
   PRUEBAS FINALIZADAS CON ÉXITO
================================================
```

Fig. 1. Complete execution of all experiments using the main orchestrator script (`run_all.py`). Part 1 covers tasks 1–3 (baseline, multiprocessing, MPI); Part 2 covers tasks 4–5 (sparse analysis and Strassen variants).

II-B Results

a) Baseline Serial Multiplication

The serial implementation confirmed the expected $\mathcal{O}(n^3)$ complexity. For a matrix of size 256×256, the execution time reached **2.5880 s**.

TABLE I
BASELINE EXECUTION TIME

Size ($n \times n$)	Execution Time (s)	Correctness
64	0.0563	True
128	0.3238	True
256	2.5880	True

b) Multiprocessing and Memory Partitioning

Using a 512×512 matrix with 10 workers for row and column partitioning, and 9 workers for block partitioning, column-based partitioning showed a performance advantage over the other strategies. All parallel strategies used Python's native `multiprocessing` library [5].

TABLE II
MULTIPROCESSING PERFORMANCE (512×512)

Partition Method	Workers	Execution Time (s)
Rows	10	0.2102
Columns	10	0.1425
Blocks (2D)	9	0.2075

c) Distributed Memory (MPI)

A distributed memory scheme was implemented using OpenMPI via `mpi4py` [12]. Executing a 512×512 matrix multiplication distributed across 4 workers resulted in an execution time of **0.0041 s**. This value includes the overhead of serializing and passing messages through MPI (`bcast`, `scatter`, `gather`).

d) Strassen's Algorithm

The hybrid approach [2], which switches to NumPy's BLAS-optimized multiplication at a threshold of 128, outperformed the pure recursive version by a factor of nearly $27\times$, reducing the overhead of recursive function calls and dynamic sub-matrix allocation.

TABLE III
STRASSEN VS. BASELINE PERFORMANCE (512×512)

Method	Threshold	Time (s)
Pure Strassen	16	0.0927
Hybrid Strassen	128	0.0034
NumPy Baseline	N/A	0.0013

e) Sparse Matrix Analysis

Two real-world matrices from the SuiteSparse Matrix Collection were processed using Compressed Sparse Row (CSR) format, applying a naive row-based load division across 4 parallel processes.

TABLE IV
SPARSE MATRICES PERFORMANCE AND SPARSITY

Matrix	Source	Sparsity (%)	CSR Time (s)
`bcspwr08`	Power Topology	99.77	0.0004
`bcsstk18`	Structural Engineering	99.90	0.0131

II-C Discussion

Data Locality and Cache Memory: The multiprocessing results demonstrate that in HPC, data organization is as critical as the number of available cores. Column partitioning (0.1425 s) significantly outperformed row partitioning (0.2102 s), which was nearly tied with block partitioning (0.2075 s). This occurs because the underlying vectorized routines operate with higher efficiency when accessing contiguous memory blocks. Maximizing sequential column access increases the processor's cache hit rate, minimizing costly reads from main RAM.

Communication Overhead in Local Environments (MPI): The exceptionally low MPI execution time (0.0041 s) initially appears counter-intuitive, as distributed memory models typically introduce bottlenecks from data serialization and network latency. However, upon detecting that all processes reside within the same container or physical machine, the MPI library bypasses network simulation and directly leverages the operating system's shared memory (`/dev/shm`). This allows highly optimized, hardware-level local transfers that negate projected network overhead. Some serialization overhead persists, making this result an empirical data point representative of intra-node MPI performance.

The Hidden Cost of Theoretical Complexity (Strassen): Although Strassen's algorithm reduces asymptotic complexity to $\mathcal{O}(n^{2.807})$ [1], the pure recursive implementation (0.0927 s) was far slower than the hybrid approach (0.0034 s). This highlights a key limitation in high-level languages like Python: call stack management and dynamic memory allocation for sub-matrices introduce massive administrative overhead. Even the well-optimized hybrid approach could not match the vectorized native C/BLAS implementation of NumPy (0.0013 s) by a factor of approximately $3\times$.

Sparsity and Load Imbalance: Using CSR format collapses the actual workload dramatically by ignoring over 99.9% of the matrix content. However, naive load division exposed the fundamental flaw of static parallelization: Worker 0 received only 20.9% of the workload while Worker 2 was assigned 26.8% of the non-zero operations (`bcsstk18` [3], `bcspwr08` [4]). In synchronous parallel architectures, execution time is strictly dictated by the slowest node, leading to worker starvation and suboptimal speedup. This concludes that geometric parallel strategies are highly inefficient for real-world sparse structures; a robust design must implement dynamic load balancers based on actual non-zero element concentration.

III Exercise 2: Parallel Cell Image Processing and Morphological Characterization

III-A Methodology and Implementation

The objective of this exercise was to implement a parallel image processing pipeline for morphological characterization using the *DIC-C2DH-HeLa* dataset. The workflow was designed to perform instance segmentation followed by geometric feature extraction. The pipeline was successfully containerized using Docker to ensure environment parity across different operating systems.

The implementation is divided into three functional modules:

1) **Data Acquisition:** A script that automates the retrieval and extraction of TIFF sequences from the Cell Tracking Challenge servers.
2) **Segmentation:** Utilizing the *Cellpose* library (version 4.0.1+) [6], which employs deep learning models to define precise cell boundaries.
3) **Orchestration & Analysis:** A module that uses *OpenCV* and *SciPy* to compute morphological metrics including cross-sectional area, major axis, and minor axis, outputting results to `morphology_summary.csv`.

A critical technical challenge was encountered when running the `multiprocessing` library within a Dockerized Linux container on **Apple Silicon** architecture. Concurrent memory locks occurred when multiple workers attempted to load the **1.15 GB** Cellpose model simultaneously. To resolve this, the multiprocessing context was explicitly set to `spawn` to ensure a clean memory space for each worker, preventing deadlocks caused by concurrent memory locks when loading the deep learning model.

```python
import multiprocessing as mp
from multiprocessing import Pool
from src.segmentation import CellAnalyzer

def worker_task(path):
    analyzer = CellAnalyzer()
    return analyzer.process_image(path)

def process_parallel(image_paths, n_workers):
    # Forced 'spawn' to avoid memory locks with heavy DL models
    context = mp.get_context('spawn')
    with context.Pool(processes=n_workers) as pool:
        results = pool.map(worker_task, image_paths)
    return results
```

Listing 1. Implementation of the spawn context for process parallelization

III-B Results and Performance Analysis

Biological validation confirmed structural coherence, with average cell areas ranging between **337 and 442** μm^2, aligning with the HeLa dataset's [7] physical resolution of $0.19\ \mu m/px$.

Computational performance was evaluated by comparing serial execution against a parallel execution utilizing 2 independent workers on a 10-core host machine. Table V details the execution times for a sample of 10 images.

TABLE V
SERIAL VS. PARALLEL PERFORMANCE COMPARISON

Execution Mode	Workers	Time (s)	Speedup (S)
Serial	1	246.28	1.00x
Parallel	2	298.58	0.82x

The speedup ($S = T_{\text{serial}}/T_{\text{parallel}}$) resulted in **0.82x**, indicating performance degradation (anti-speedup). Efficiency ($E = S/N$) dropped to 0.41.

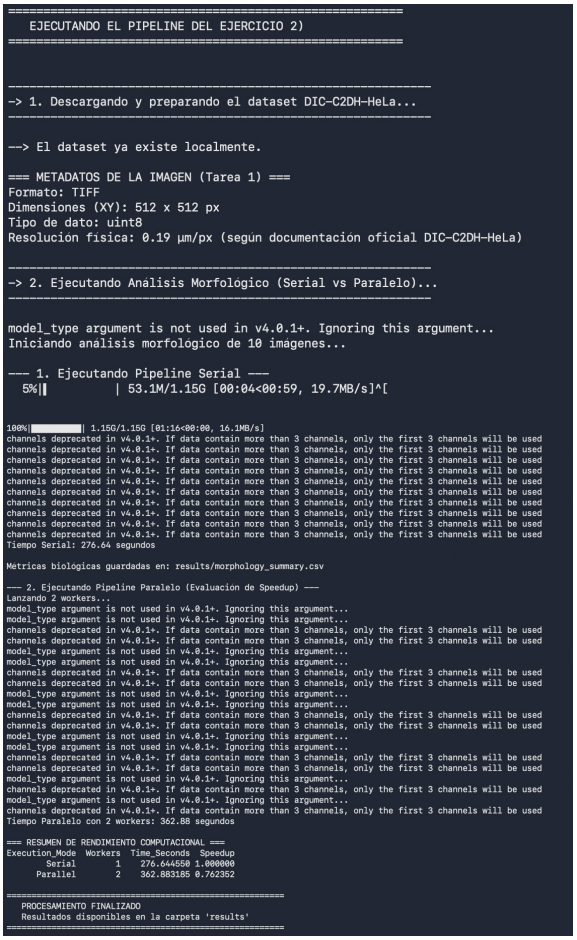

Fig. 2. Terminal output showing data acquisition, metadata inspection, segmentation process, and morphology summary generation.

III-C Discussion

The results highlight a phenomenon commonly encountered in HPC known as **anti-speedup** (speedup < 1.0x), where the parallel execution is slower than the serial baseline.

This behavior is entirely explained by the initialization overhead inherent to the system architecture. In the parallel strategy using the `spawn` context, each worker process starts a clean Python interpreter and independently loads the 1.15 GB Cellpose neural network weights into memory. Because the test dataset contained only 10 images, the time spent allocating

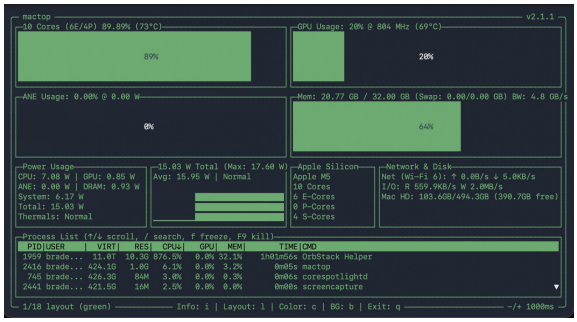

Fig. 3. CPU resource monitor during parallel execution. The high initial load corresponds to simultaneous model loading across worker processes.

memory and resolving network I/O contention overshadowed the time spent actively predicting cell boundaries.

A significant limitation is the **memory bandwidth bottleneck**: when multiple isolated processes attempt to load a gigabyte-scale model simultaneously within a constrained virtualized environment like Docker, the hardware spends idle cycles resolving locks rather than executing tensor operations.

The parallel strategy is highly advantageous only when the computational workload heavily outweighs the startup cost. If the dataset contained 1,000 images instead of 10, the initialization overhead would be diluted across a longer execution time, likely resulting in a positive speedup closer to theoretical expectations. For small workloads involving large neural network weights, serial execution remains the most efficient technical decision.

IV Exercise 3: Forest Fire Cellular Automaton Driven by NASA FIRMS Data

IV-A Introduction

Cellular automata are widely used for modeling complex spatiotemporal systems such as forest fire propagation. In this exercise, real satellite-based fire detection data from NASA FIRMS [8] is integrated to initialize a discrete fire spread model over a selected region in West Africa (Mauritania and Mali), an area consistently ranked among the most fire-affected regions globally [11]. The objective is to analyze both the dynamics of fire propagation and the computational performance of a parallel implementation using MPI.

IV-B Methodology

1) Forest Fire Model

The state space $\mathcal{S}$ for each cell (i, j) is defined as $\mathcal{S} \in \{0, 1, 2, 3\}$, representing *non-burnable*, *vegetation*, *burning*, and *burned* states, respectively. Fire propagation follows Moore neighborhood interactions (8 neighbors), where a vegetation cell ignites with probability:

$$P(\text{ignite}) = p_{\text{spread}} = 0.45 \tag{1}$$

if at least one neighboring cell is in the burning state. Burning cells transition to the burned state after one iteration.

2) NASA FIRMS Data Integration

Hotspot data was retrieved from the NASA FIRMS API [9], [10] using the `VIIRS_NOAA20_NRT` instrument (Near Real-Time), covering a two-day window ending on April 23, 2026. The selected bounding box spans latitude 18–28°N and longitude 14–4°W. A total of 768 hotspots were retrieved without confidence threshold filtering. Each hotspot is mapped to the simulation grid via linear interpolation:

```
col = int((r["longitude"] - west) / (east - west) * (cols - 1))
row = int((north - r["latitude"]) / (north - south) * (rows - 1))
```

Listing 2. Coordinate mapping from FIRMS hotspots to simulation grid

Mapped cells are initialized as burning (state 2); all remaining cells are initialized as susceptible vegetation (state 1). It is important to note that FIRMS hotspot detections represent thermal anomalies rather than exact fire perimeters, so the initialized burning cells indicate probable ignition locations but do not capture the spatial extent of actual fires.

3) Parallel Implementation

The computational domain is decomposed into horizontal subdomains, where each MPI process is assigned a contiguous block of rows via `numpy.array_split`. The implementation uses `mpi4py` [12]. To maintain consistency across subdomain boundaries, ghost rows are exchanged between neighboring processes at each iteration using `MPI.Sendrecv`, ensuring fire spreads correctly across process boundaries. The overall grid is reconstructed via `MPI.gather` on rank 0. Execution time is measured using `MPI.Wtime()` for the parallel version and `time.time()` for the serial version, both excluding I/O and visualization.

IV-C Results

1) Runtime Comparison

TABLE VI
EXECUTION TIME COMPARISON ACROSS GRID SIZES AND PROCESS COUNTS

Method	Grid Size	Processes	Time (s)
Serial	500×500	1	0.4238
Serial	1000×1000	1	1.9239
Serial	1500×1500	1	4.4600
MPI	1500×1500	2	3.2260
MPI	1500×1500	4	2.0400

Serial runtimes scale roughly quadratically with grid side length, consistent with $\mathcal{O}(N^2)$ complexity per time step. The speedups for the 1500×1500 grid are:

$$S_4 = \frac{T_1}{T_4} = \frac{4.46}{2.04} \approx 2.19\text{x}, \quad S_2 = \frac{4.46}{3.226} \approx 1.38\text{x} \tag{2}$$

```
Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 517
Serial runtime: 0.4238 sec
Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 601
Serial runtime: 1.9239 sec
```

```
Loading NASA FIRMS data...
VIIRS_NOAA20_NRT: 768 hotspots
Initial fire cells: 641
Execution time: 3.226 sec
Simulation finished.
```

Fig. 4. Execution logs for serial and MPI configurations confirming 768 FIRMS hotspots and showing execution times for 500^2, 1000^2, and 1500^2 grids.

2) Temporal Evolution and Spatial Dynamics

All visualizations were generated using a representative grid of 800×800 cells and 140 time steps with `VISUAL = True`, isolating rendering overhead from the performance benchmarks above.

a) Early Expansion (Step 10)

Fire fronts expand radially from the 554 initial burning cells (mapped from 768 FIRMS detections), forming characteristic circular patches. Large contiguous regions of vegetation remain unaffected.

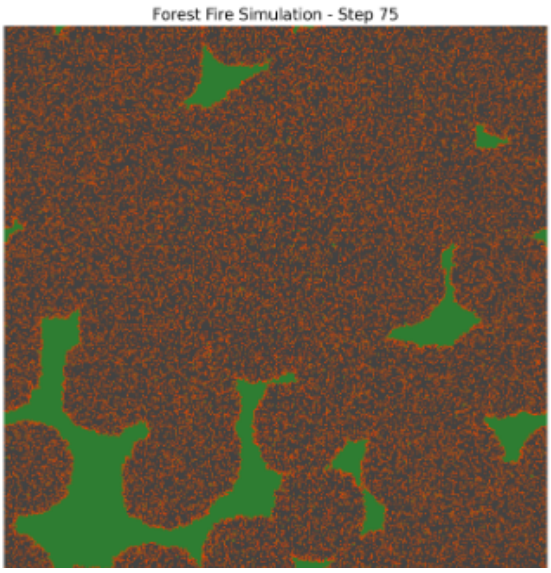

Fig. 5. Step 10: Early expansion phase. Independent fire clusters grow outward from FIRMS hotspot locations; most of the domain remains vegetated (green).

b) Mid-Propagation and Merging (Step 40)

Fire fronts from neighboring clusters begin to merge, creating continuous burned corridors. Vegetation fragmentation becomes evident. In a parallel context, this phase represents the highest synchronization load between MPI processes as the active fire front crosses horizontal domain boundaries.

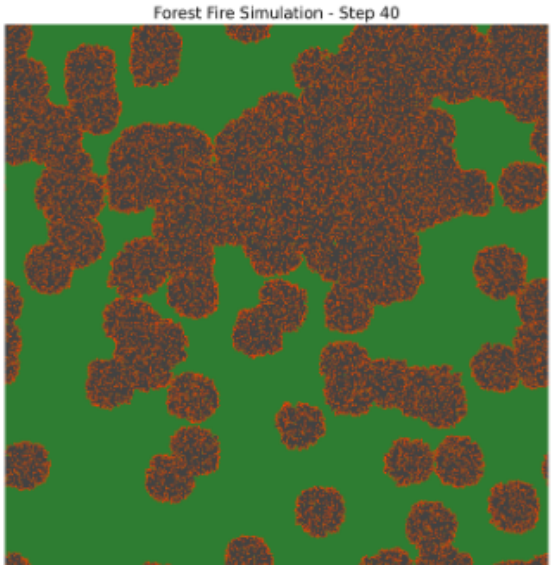

Fig. 6. Step 40: Mid-propagation phase. Fire fronts from adjacent clusters merge; vegetation (green) becomes increasingly fragmented.

c) Late Stage and Stationarity (Steps 75–140)

At step 75, most of the grid has transitioned to the burned state (dark gray). Surviving vegetation is confined to isolated islands protected by burned firebreaks. By step 85 the system reaches full stationarity with no remaining active fire cells.

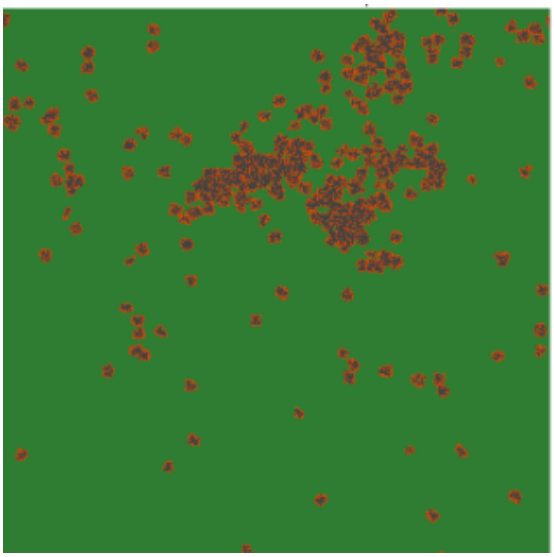

Fig. 7. Step 75: Late propagation stage. Most of the domain is burned; surviving vegetation is confined to isolated islands protected by burned firebreaks.

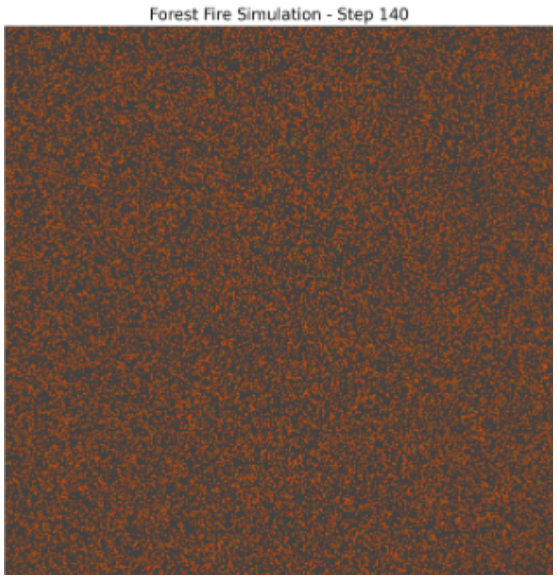

Fig. 8. Step 140: Final stationary state. Nearly all vegetation has been consumed; the domain is dominated by burned cells (dark gray) with sparse residual vegetation.

IV-D Discussion

The experiment demonstrates that domain decomposition is an effective strategy for accelerating cellular automata. The MPI implementation successfully reduced runtimes; however, efficiency ($E = S/N$) was limited to approximately 0.55 for 4 processes. This is attributed to the **communication-to-computation ratio**: at each step, processes must synchronize ghost rows, introducing latency that grows more significant as the number of processes increases.

The temporal evolution is consistent with expected fire dynamics: isolated ignition points expand into continuous fire fronts, which eventually exhaust available fuel. The survival of isolated vegetation patches in the final state is a well-known emergent property of probabilistic fire automata, arising when burned regions create barriers that interrupt propagation.

IV-E Limitations

The model simplifies real fire dynamics by assuming homogeneous vegetation, constant ignition probability, and purely local interactions. Environmental factors such as wind, relative humidity, terrain elevation, and fuel heterogeneity are not considered. The use of FIRMS detections without confidence filtering may include low-quality observations. Spatial discretization of hotspot coordinates also introduces quantization error.

Future improvements could include incorporating Fire Radiative Power (FRP) values as local ignition probability modifiers, wind-driven anisotropic spread, and confidence-based filtering of detections.

V Exercise 4: Parallel K-Means Clustering

V-A Methodology

K-Means is a computationally intensive unsupervised learning algorithm whose cost scales with the number of observations N, dimensions D, and clusters K [13]. For this exercise, a distributed-memory implementation was developed using `mpi4py` [12], [14] to process the **Covertype dataset** (581,012 samples, 54 features).

The workload is distributed using a **Data Decomposition** strategy:

1) **Data Partitioning:** The dataset is divided into local observation blocks using `Scatterv`, which correctly handles remainder rows when the dataset size is not perfectly divisible by the number of workers.
2) **Normalization:** `StandardScaler` is applied to prevent features with high variance from dominating Euclidean distance calculations.
3) **Local Assignment:** Each MPI process independently calculates the Euclidean distance between its local points and the global centroids.
4) **Global Synchronization:** Centroid updates are synchronized globally at each iteration using `comm.Allreduce`, allowing all processes to compute new global centroids simultaneously without a centralized bottleneck.

V-B Evidence of Completeness

The implementation was successfully containerized and executed. A key technical adaptation was the explicit casting of labels to `int32` before using `Gatherv` to ensure compatibility with MPI's internal message buffer constraints and avoid message truncation errors.

```
# Ensuring MPI compatibility to avoid message truncation errors
local_labels_32 = local_labels.astype(np.int32)
all_counts = comm.gather(len(local_labels_32), root=0)

if rank == 0:
    global_labels = np.empty(sum(all_counts), dtype=np.int32)
    displs = [sum(all_counts[:i]) for i in range(size)]
    comm.Gatherv(sendbuf=local_labels_32,
                 recvbuf=[global_labels, all_counts, displs, MPI.INT],
                 root=0)
else:
    comm.Gatherv(sendbuf=local_labels_32, recvbuf=None, root=0)
```

Listing 3. Optimized Gatherv implementation with int32 casting

Fig. 9. Complete execution output showing parallel K-Means convergence logs and benchmark results for $K = 7$ and $K = 10$.

V-C Results

The algorithm was benchmarked using $K = 7$ and $K = 10$ clusters across different worker configurations. The serial baseline recorded an execution time of **245.38 s** for 50 iterations.

V-D Discussion

Communication vs. Computation: The primary scalability bottleneck is the *Communication-to-Computation Ratio*. While parallelization accelerates distance calculations ($\mathcal{O}(N/P)$ per process), collective operations like `Allreduce` introduce $\mathcal{O}(\log P)$ latency. For $K = 7$, increasing processes from 2 to 4 raised the time per iteration from 4.60 s to 6.80 s, indicating communication costs began to overshadow computational gains. When K increased to 10, the message payload in collective operations grew by 42%, resulting in higher total runtimes.

TABLE VII
K-MEANS PERFORMANCE METRICS (COVERTYPE DATASET)

K	Mode	Iter.	Total Time (s)	Time/Iter (s)	Speedup
7	Serial	50	245.38	4.90	1.00x
7	MPI (1)	8	48.17	6.02	5.09x*
7	MPI (2)	4	18.41	4.60	13.32x*
7	MPI (4)	11	74.85	6.80	3.27x
10	MPI (1)	15	134.93	8.99	1.81x
10	MPI (4)	18	131.50	7.30	1.86x

**Speedup is sensitive to centroid initialization; lower iteration counts lead to inflated speedup values. Time per Iteration is a more stable comparative metric.*

Initialization Sensitivity: The high variability in total time is largely due to random starting positions of centroids. In the $K = 7$ MPI (2) run, the algorithm converged in only 4 iterations, yielding an outlier speedup. This highlights the importance of using *time per iteration* as a more stable metric for parallel benchmarking.

Beneficial Conditions: The parallel strategy is most advantageous when the dataset exceeds the memory limits of a single node, or when the number of samples N is large enough that the $\mathcal{O}(N/P)$ reduction in computation significantly outweighs the $\mathcal{O}(\log P)$ communication overhead of MPI collectives.

VI Conclusion

This report analyzed four critical applications of high-performance computing: *matrix multiplication, microscopy image processing, forest fire cellular automata, and parallel K-Means clustering*.

Across all exercises, the results demonstrate that **parallelism is not a guaranteed benefit**. Its effectiveness is strictly governed by the balance between computational workload and overhead (communication, model loading, or synchronization). Specifically:

1) Large-scale mathematical and grid-based operations (Exercises 1 & 3) benefit significantly from domain decomposition, with MPI effectively reducing runtimes at the cost of sub-linear scaling due to communication overhead.
2) Deep learning-based pipelines (Exercise 2) face heavy anti-speedup due to resource contention during model loading; parallelism only becomes beneficial when the dataset size is large enough to amortize startup costs.
3) Distributed machine learning (Exercise 4) scales well until the latency of global synchronization (`Allreduce`) outweighs local calculation time, with initialization sensitivity complicating direct speedup comparisons.

Ultimately, successful HPC implementation requires a deep understanding of the underlying hardware architecture, the

memory access patterns of the algorithm, and the specific communication patterns introduced by the chosen parallelization strategy.

References

[1] Naukri Code 360, "Strassen's Matrix Multiplication," *Coding Library*. [Online]. Available: https://www.naukri.com/code360/library/strassens-matrix-multiplication

[2] A. Sharma et al., "Hybrid optimization technique for matrix chain multiplication using Strassen's algorithm," *PMC*, 2025. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12000801/

[3] J. Lewis, "bcsstk18: Stiffness matrix, small power system model," *SuiteSparse Matrix Collection*, HB Group. [Online]. Available: https://sparse.tamu.edu/HB/bcsstk18

[4] B. Dembart and J. Lewis, "bcspwr08: Pattern of the BC Hydro power system," *SuiteSparse Matrix Collection*, HB Group. [Online]. Available: https://sparse.tamu.edu/HB/bcspwr08

[5] Python Software Foundation, "multiprocessing — Process-based parallelism," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/multiprocessing.html

[6] C. Stringer, T. Wang, M. Michaelos, and M. Pachitariu, "Cellpose: a generalist algorithm for cellular segmentation," *Nature Methods*, vol. 18, pp. 100–106, 2021.

[7] *Cell Tracking Challenge: DIC-C2DH-HeLa Dataset*. [Online]. Available: http://celltrackingchallenge.net/2d-datasets/

[8] NASA Fire Information for Resource Management System (FIRMS), *FIRMS Main Portal*. [Online]. Available: https://firms.modaps.eosdis.nasa.gov/

[9] NASA FIRMS, *FIRMS API*. [Online]. Available: https://firms.modaps.eosdis.nasa.gov/api/

[10] NASA FIRMS Academy, *FIRMS API Python Guide*. [Online]. Available: https://firms.modaps.eosdis.nasa.gov/content/academy/data_api/firms_api_use.html

[11] World Resources Institute, *Global Trends in Forest Fires*. [Online]. Available: https://www.wri.org/insights/global-trends-forest-fires

[12] mpi4py Developers, *mpi4py — MPI for Python Documentation*. [Online]. Available: https://mpi4py.readthedocs.io/

[13] T. Kanungo et al., "An efficient k-means clustering algorithm: analysis and implementation," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2002.

[14] P. S. Pacheco, *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.